

哈尔滨工业大学 × 法国里昂商学院

HIT × emlyon business school

Python Basic Syntax

Tutorial

Python 基础语法教程（精简版）

黄子扬 Ziyang Huang

June 13, 2026

Contents

1	Variables & Data Types / 变量与数据类型	2
1.1	Hello World & Basic Output	2
1.2	Variables & Assignment / 变量与赋值	2
1.3	Basic Data Types / 基本数据类型	2
1.4	The None Type / 空值	3
1.5	Type Casting / 类型转换	3
1.6	User Input / 用户输入	4
2	Operators / 运算符	5
2.1	Arithmetic Operators / 算术运算符	5
2.2	String Operators / 字符串特殊运算	5
2.3	Comparison Operators / 比较运算符	5
2.4	Logical Operators / 逻辑运算符	6
3	Strings / 字符串完全指南	7
3.1	String Creation / 创建字符串	7
3.2	Indexing / 索引	7
3.3	Slicing / 切片	7
3.4	String Length / 字符串长度	7
3.5	Escape Characters / 转义字符	8
3.6	String Methods / 字符串常用方法	8
3.7	String Formatting / 字符串格式化	9
4	Control Flow / 控制流程	10
4.1	Conditionals: if / elif / else / 条件判断	10
4.2	Ternary Operator / 三元表达式 (一行 if/else)	10
4.3	For Loops / For 循环	10
4.4	While Loops / While 循环	11
4.5	Loop Control Keywords / 循环控制关键字	11
4.6	Loop Utilities / 循环工具	11
5	Functions / 函数	13

5.1	Defining Functions / 定义函数	13
5.2	Scope / 作用域	13
5.3	Lambda Functions / 匿名函数	14
6	Data Structures / 数据结构	15
6.1	List / 列表 []	15
6.2	Tuple / 元组 ()	16
6.3	Dictionary / 字典 {}	16
6.4	Set / 集合 {}	17
6.5	Comprehensions / 推导式	17
7	Common Built-in Functions / 常用内置函数	19
8	Quick Reference Card / 速查卡	20

1 Variables & Data Types / 变量与数据类型

1.1 Hello World & Basic Output

Your First Python Program / 你的第一个 Python 程序

```
1 print("Hello, World!")      # Output: Hello, World!
```

`print()` 是 Python 最基础的输出函数，将括号内的内容打印到屏幕上。`#` 后面的文字是注释，Python 不会执行。

1.2 Variables & Assignment / 变量与赋值

The Assignment Operator / 赋值运算符

变量是存储数据的“容器”。Python 使用 `=` 将右侧的值绑定到左侧的变量名上：

```
1 age = 25                    # 将 25 赋给变量 age
2 name = "Alice"             # 将 "Alice" 赋给变量 name
3 print(age)                  # 25
4 print(name)                 # Alice
```

Variable Naming Rules / 变量命名禁忌

- 不能数字开头: `3text = 4` ——会报 `SyntaxError`
- 不能用保留字: 不要用 `print`, `for`, `in`, `list`, `str`, `int` 做变量名
- 大小写敏感: `age` 和 `Age` 是两个不同的变量
- 命名规范: 使用 `lower_case_with_underscores` (蛇形命名法)，如 `car_length`

1.3 Basic Data Types / 基本数据类型

Four Primitive Types / 四种原始类型

Type / 类型	Keyword	Example / 示例	Description / 说明
Integer / 整数	<code>int</code>	100	整数，可正可负
Float / 浮点数	<code>float</code>	10.7	小数（带小数点）
String / 字符串	<code>str</code>	"Hello"	引号内的文本
Boolean / 布尔值	<code>bool</code>	True / False	真/假（首字母大写）

用 `type()` 函数可以查看任何变量的类型：

```
1 print(type(100))           # <class 'int'>
2 print(type(10.7))          # <class 'float'>
3 print(type("Hello"))       # <class 'str'>
4 print(type(True))           # <class 'bool'>
```

1.4 The None Type / 空值

None — "No Value" / 空值

`None` 表示“没有值”。它不等于 0、`False` 或空字符串 `""`，`None` 是 `NoneType` 类型，只等于它自己。

```
1 my_var = None
2 print(type(my_var))          # <class 'NoneType'>
```

1.5 Type Casting / 类型转换

Converting Between Types / 类型之间互转

```
1 print(int(9.9))             # 9 —— 截断（不四舍五入！）
2 print(float(5))              # 5.0
3 print(str(100))              # '100'
4 print(bool(1))               # True
5 print(bool(0))               # False（0、空字符串、None、空容器都是 False）
```

String + Int = Error! / 字符串 + 整数 = 报错！

```
1 a = 34
2 b = "10"
3 # print(a + b)              # TypeError! 不能将 int 和 str 直接相加
4 print(a + int(b))           # 44 —— 先转换再运算
```

1.6 User Input / 用户输入

Always Returns String / input() 总是返回字符串

```
1 name = input("Enter your name: ")    # 用户输入
2 print("Hello, " + name)              # 输出: Hello, [用户输入的名字]
```

重要: `input()` 的返回值永远是 `str` 类型。如果要做数学运算，必须先用 `int()` 或 `float()` 转换。

2 Operators / 运算符

2.1 Arithmetic Operators / 算术运算符

Operator	Name / 名称	Example / 示例	Result / 结果
+	Addition / 加法	2 + 3	5
-	Subtraction / 减法	5 - 2	3
*	Multiplication / 乘法	3 * 2	6
/	Division / 除法	5 / 2	2.5 (总是返回 float)
//	Floor Division / 整除	5 // 2	2 (丢弃余数)
**	Exponent / 幂	3 ** 2	9
%	Modulo / 取余	7 % 3	1

2.2 String Operators / 字符串特殊运算

Concatenation / Repetition / 拼接与重复

```
1 print("Hello" + " World")    # 'Hello World' —— 拼接
2 print("Ha" * 3)              # 'HaHaHa' —— 重复
```

2.3 Comparison Operators / 比较运算符

Operator	Meaning / 含义	Example / 示例
==	Equal / 等于	1 == 1 → True
!=	Not equal / 不等于	2 != 3 → True
<	Less than / 小于	2 < 5 → True
>	Greater than / 大于	5 > 3 → True
<=	Less than or equal / 小于等于	3 <= 3 → True
>=	Greater than or equal / 大于等于	5 >= 5 → True

比较运算的结果永远是 `bool` 类型 (`True` 或 `False`)。

2.4 Logical Operators / 逻辑运算符

Operator	Rule / 规则	Example / 示例
<code>and</code>	两边都为 <code>True</code> 才返回 <code>True</code>	<code>True and False</code> $\rightarrow$ <code>False</code>
<code>or</code>	任一边为 <code>True</code> 就返回 <code>True</code>	<code>True or False</code> $\rightarrow$ <code>True</code>
<code>not</code>	取反	<code>not True</code> $\rightarrow$ <code>False</code>

```
1 print((5 > 3) and (2 < 4)) # True  —— 两个条件都成立
2 print((5 > 3) or (2 > 4))  # True  —— 至少一个成立
3 print(not (5 > 3))         # False —— 取反
```


3 Strings / 字符串完全指南

3.1 String Creation / 创建字符串

Single or Double Quotes / 单引号或双引号

```
1 s1 = "Hello"
2 s2 = 'World'
3 s3 = "It's okay"      # 双引号内可含单引号
4 s4 = 'She said, "Hi!"' # 单引号内可含双引号
```

3.2 Indexing / 索引

Zero-Based Indexing / 从 0 开始计数

字符串中每个字符都有一个编号（索引），从 0 开始。负索引从右向左，-1 是最后一个字符。

```
1 text = "Python"
2 print(text[0])      # 'P' —— 第一个字符
3 print(text[1])      # 'y'
4 print(text[-1])     # 'n' —— 最后一个字符
5 print(text[-2])     # 'o'
```

3.3 Slicing / 切片

—Left-Closed, Right-Open / 左闭右开

切片格式：[start:stop]。包含 start，不包含 stop。

```
1 text = "Python BootCamp"
2 print(text[0:6])      # 'Python' —— 索引 0,1,2,3,4,5 (不含 6)
3 print(text[7:11])     # 'Boot'
4 print(text[:6])       # 'Python' —— 省略 start，默认从 0 开始
5 print(text[7:])       # 'BootCamp' —— 省略 stop，默认到末尾
6 print(text[::-1])     # 'pmaCtooB nohtyP' —— 反转字符串！
```

3.4 String Length / 字符串长度

```
1 text = "Hello World"
2 print(len(text))      # 11 —— 空格也算一个字符
```

3.5 Escape Characters / 转义字符

Special Characters / 特殊字符

Escape / 转义	Meaning / 含义
<code>\n</code>	Newline / 换行
<code>\t</code>	Tab / 制表符
<code>\\</code>	Backslash / 反斜杠本身
<code>\"</code>	Double quote / 双引号
<code>\'</code>	Single quote / 单引号

```
1 print("Line1\nLine2")      # 两行输出
2 print("Name:\tAlice")      # Name:    Alice
```

3.6 String Methods / 字符串常用方法

Strings Are Immutable / 字符串不可变

所有字符串方法都返回新字符串，原字符串不变。要保留结果必须重新赋值！

Method	Description / 说明
<code>s.lower()</code>	转小写: <code>"ABC".lower()</code> → <code>'abc'</code>
<code>s.upper()</code>	转大写: <code>"abc".upper()</code> → <code>'ABC'</code>
<code>s.strip()</code>	去除首尾空白 (<code>lstrip</code> 仅左, <code>rstrip</code> 仅右)
<code>s.replace(old, new)</code>	替换: <code>"Hi".replace("i","ello")</code> → <code>'Hello'</code>
<code>s.split(delim)</code>	拆分: <code>"a,b,c".split(",")</code> → <code>['a','b','c']</code>
<code>delim.join(list)</code>	连接: <code>"-".join(['a','b'])</code> → <code>'a-b'</code>
<code>s.count(sub)</code>	统计出现次数: <code>"ababa".count("a")</code> → <code>3</code>
<code>s.find(sub)</code>	查找位置——找不到返回 <code>-1</code> (不报错)
<code>s.startswith(p)</code>	是否以 <code>p</code> 开头 → <code>bool</code>
<code>s.endswith(p)</code>	是否以 <code>p</code> 结尾 → <code>bool</code>

```
1 name = "  Alice  "
2 print(name.strip())      # 'Alice' —— 原字符串不变!
3 name = name.strip()      # 必须重新赋值才能保留
```

```
4 print(name)                # 'Alice'
```

3.7 String Formatting / 字符串格式化

f-string (Recommended) / f-字符串 (推荐, Python 3.6+)

```
1 name = "Alice"
2 age = 20
3 print(f"{name} is {age} years old.")    # Alice is 20 years old.
4 print(f"Pi = {3.14159:.2f}")           # Pi = 3.14 (保留两位小数)
```

Other Formatting Methods / 其他格式化方法

```
1 # .format() 方法
2 print("{} is {} years old.".format(name, age))
3
4 # % 旧式格式化 (不推荐)
5 print("%s is %d years old." % (name, age))
6
7 # 逗号拼接 (print 自动加空格, 需手动 str() 转换数字)
8 print(name, "is", str(age), "years old.")
```

4 Control Flow / 控制流程

4.1 Conditionals: if / elif / else / 条件判断

The Full Structure / 完整结构

条件判断让程序根据条件选择执行不同的代码分支。**缩进**（4 个空格）决定哪些代码属于哪个分支。

```
1 price = 42
2
3 if price > 100:
4     print("expensive")           # price > 100 时执行
5 elif price > 50:                 # 仅在 price <= 100 时才检查
6     print("moderate")           # 50 < price <= 100 时执行
7 else:                           # 以上条件都不满足时
8     print("cheap")              # price <= 50 时执行
```

Execution Order / 执行顺序

Python 从上到下检查条件，第一个为 **True** 的分支被执行，其余全部跳过。

4.2 Ternary Operator / 三元表达式（一行 if/else）

```
1 # 语法: value_if_true if condition else value_if_false
2 result = "pass" if score >= 60 else "fail"
```

4.3 For Loops / For 循环

—遍历序列

```
1 # 遍历列表
2 fruits = ["apple", "banana", "cherry"]
3 for fruit in fruits:
4     print(fruit)
5
6 # 使用 range() 循环固定次数
7 for i in range(5):           # 0, 1, 2, 3, 4 (不含 5!)
8     print(i)
9
10 for i in range(1, 11):       # 1, 2, ..., 10
11     print(i)
12
13 for i in range(0, 10, 2):     # 0, 2, 4, 6, 8 (步长为 2)
14     print(i)
```

`range(start, stop, step)` 同样遵循”左闭右开”：含 `start`，不含 `stop`。

4.4 While Loops / While 循环

Repeat While Condition is True / 条件为真就重复

```
1 counter = 1
2 while counter <= 5:
3     print(counter)
4     counter = counter + 1      # 必须更新计数器！否则会无限循环
```

For vs While / 何时用哪种？

- **For**: 知道要循环多少次——遍历序列或固定次数。
- **While**: 只知道什么时候停，但不知道要循环多少次。

4.5 Loop Control Keywords / 循环控制关键字

break, continue, pass / 三大控制关键字

```
1 # break: 立即退出整个循环
2 for i in range(1, 10):
3     if i == 5:
4         break                # 退出循环
5     print(i)                 # 输出: 1, 2, 3, 4
6
7 # continue: 跳过本次迭代, 进入下一次
8 for i in range(1, 10):
9     if i == 5:
10        continue            # 跳过 i=5
11    print(i)                 # 输出: 1,2,3,4,6,7,8,9 (没有 5)
12
13 # pass: 占位符——什么都不做
14 for i in range(1, 10):
15     if i == 5:
16         pass                # 什么都不做
17    print(i)                 # 输出: 1,2,3,4,5,6,7,8,9
```

4.6 Loop Utilities / 循环工具

```
1 # enumerate: 同时获取索引和值
2 for i, fruit in enumerate(["a", "b", "c"]):
3     print(i, fruit)          # 0 a, 1 b, 2 c
4
5 # zip: 并行遍历多个序列
6 for a, b in zip([1, 2, 3], ["x", "y", "z"]):
```

```
print(a, b)          # 1 x, 2 y, 3 z
```

5 Functions / 函数

5.1 Defining Functions / 定义函数

The Keyword / def 关键字

函数是可重复使用的代码块。`def` 关键字定义函数，`return` 返回值。

```
1 def multiply(x, y):           # 函数签名：名称 + 参数
2     result = x * y           # 函数体（缩进！）
3     return result            # 返回结果
4
5 # 调用函数
6 answer = multiply(3, 4)       # answer = 12
7 print(answer)                 # 12
```

Key Terminology / 关键术语

- **Parameter / 形参**：函数定义时的变量名（如 `x`, `y`）
- **Argument / 实参**：调用函数时传入的实际值（如 `3`, `4`）
- **Return Value / 返回值**：函数执行完返回的结果——无 `return` 则返回 `None`

5.2 Scope / 作用域

Local vs Global / 局部变量 vs 全局变量

函数内部定义的变量是**局部变量**，只在函数内部可见。函数外部定义的变量是**全局变量**。

```
1 x = 5                         # 全局变量
2
3 def func():
4     y = 10                     # 局部变量（仅 func 内部可见）
5     x = 99                     # 创建新的局部 x——不影响全局 x!
6     print("Inside:", x)        # 99（局部 x）
7
8 func()
9 print("Outside:", x)           # 5（全局 x 没变！）
10 # print(y)                    # NameError! y 只在 func 内部存在
```

读取全局变量可以在函数内部直接做；**修改**全局变量需要用 `global` 关键字声明。

5.3 Lambda Functions / 匿名函数

One-Line Anonymous Functions / 一行匿名函数

`lambda` 创建不需要名字的简单函数，只能包含单个表达式：

```
1 # 传统写法
2 def add(x, y):
3     return x + y
4
5 # lambda 写法
6 add = lambda x, y: x + y
7
8 print(add(3, 4))           # 7
9
10 # lambda 常用于排序、过滤等场景
11 pairs = [(1, 'one'), (2, 'two'), (3, 'three')]
12 pairs.sort(key=lambda p: p[1]) # 按第二个元素（字符串）排序
```


6 Data Structures / 数据结构

6.1 List / 列表 []

Mutable, Ordered Sequence / 可变、有序序列

列表是最常用的数据结构——用方括号 [] 创建，元素可变，保持插入顺序。

```
1 # 创建
2 fruits = ["apple", "banana", "cherry"]
3 mixed = [1, "hello", 3.14, True]          # 可混合不同类型
4
5 # 索引和切片（和字符串一样）
6 print(fruits[0])                        # 'apple'
7 print(fruits[-1])                       # 'cherry'
8 print(fruits[0:2])                      # ['apple', 'banana']
```

List Methods / 列表常用方法

Method	Description / 说明
lst.append(x)	末尾追加元素
lst.extend(iter)	批量追加（从另一个可迭代对象）
lst.insert(i, x)	在位置 i 插入
lst.remove(x)	删除首次出现的 x——找不到则报错
lst.pop(i)	弹出位置 i 的元素（默认最后一个）并返回
lst.clear()	清空列表
lst.index(x)	查找首次出现的位置
lst.count(x)	统计出现次数
lst.sort()	原地排序 ——修改原列表，返回 None
sorted(lst)	返回新排序列表 ——原列表不变
lst.reverse()	原地反转
lst.copy()	浅拷贝（等价于 <code>lst[:]</code> ）
len(lst)	列表长度

```
1 nums = [3, 1, 2]
2 nums.append(4)          # [3, 1, 2, 4]
```

```
3 nums.sort()           # [1, 2, 3, 4] —— 原地排序
4 print(nums.pop())     # 4 —— 弹出并返回最后一个元素
5 print(nums)           # [1, 2, 3]
```

The Reference Pitfall / 引用陷阱（重要考点）

```
1 a = [1, 2, 3]
2 b = a                 # b 和 a 指向同一个列表！
3 b.append(4)
4 print(a)              # [1, 2, 3, 4] —— a 也被改了！
5
6 # 正确拷贝方式
7 c = a.copy()          # 或 c = a[:] 或 c = list(a)
8 c.append(5)
9 print(a)              # [1, 2, 3, 4] —— 不受影响
```

= 只创建引用（别名），不是真正的复制！

6.2 Tuple / 元组 ()

Immutable, Ordered Sequence / 不可变、有序序列

元组和列表类似，但**不可变**——创建后不能修改、添加或删除元素。

```
1 # 创建
2 point = (3, 4)
3 person = ("Alice", 20, "Student")
4
5 # 索引和切片（和列表一样）
6 print(point[0])       # 3
7
8 # 解包（Unpacking）
9 x, y = point
10 print(x, y)          # 3 4
11
12 # 单元素元组需要逗号
13 single = (1,)        # 注意逗号！（1）只是带括号的整数
```

6.3 Dictionary / 字典 {}

Key-Value Pairs / 键值对——通过键来查找值

字典存储**键值对**，通过唯一的键来访问值（而不是靠位置索引）。

```
1 # 创建
2 student = {"name": "Alice", "age": 20, "grade": "A"}
3
4 # 访问
5 print(student["name"]) # 'Alice'
```

```

6 print(student.get("age"))    # 20 —— 推荐! 键不存在返回 None 不报错
7 # student["height"]        # KeyError! 键不存在
8
9 # 增删改
10 student["height"] = 170    # 添加新键值对
11 student["age"] = 21        # 修改已有键的值
12 del student["grade"]       # 删除键值对
13
14 # 遍历
15 for key, value in student.items():
16     print(f"{key}: {value}")
17
18 # 检查键是否存在
19 if "name" in student:
20     print("Found!")

```

6.4 Set / 集合 {}

Unordered, Unique Elements / 无序、不重复

集合自动去重，支持数学集合运算（交、并、差）。

```

1 # 创建（空集合必须用 set(), {} 是空字典!）
2 colors = {"red", "blue", "green"}
3 colors.add("yellow")        # 添加
4 colors.remove("red")        # 删除——不存在则报错
5 colors.discard("purple")    # 安全删除——不存在也不报错
6
7 # 集合运算
8 A = {1, 2, 3, 4}
9 B = {3, 4, 5, 6}
10 print(A | B)                # {1,2,3,4,5,6} —— 并集
11 print(A & B)                # {3,4} —— 交集
12 print(A - B)                # {1,2} —— 差集
13
14 # 去重
15 duplicates = [1, 2, 2, 3, 3, 3]
16 unique = list(set(duplicates)) # [1, 2, 3]

```

6.5 Comprehensions / 推导式

One-Line Loops / 一行搞定循环

推导式是 for 循环的简洁写法，用于快速创建列表、字典、集合。

```

1 # 列表推导式
2 squares = [x**2 for x in range(1, 6)]    # [1, 4, 9, 16, 25]
3
4 # 带条件的列表推导式

```

```
5 evens = [x for x in range(1, 11) if x % 2 == 0] # [2, 4, 6, 8, 10]
6
7 # 字典推导式
8 sq_dict = {x: x**2 for x in range(1, 6)}          # {1:1, 2:4, 3:9, 4:16, 5:25}
9
10 # 集合推导式
11 sq_set = {x**2 for x in [1, -1, 2, -2, 3, -3]}    # {1, 4, 9} —— 自动去重!
```

7 Common Built-in Functions / 常用内置函数

Function	Description / 说明
<code>print(x)</code>	输出到控制台
<code>input(prompt)</code>	读取用户输入——总是返回 <code>str</code>
<code>type(obj)</code>	返回数据类型
<code>len(seq)</code>	返回序列长度（字符串、列表、元组等）
<code>range(start, stop, step)</code>	生成整数序列——左闭右开
<code>sum(iterable)</code>	求和
<code>max(iterable)</code>	最大值
<code>min(iterable)</code>	最小值
<code>abs(x)</code>	绝对值
<code>round(x, n)</code>	保留 <code>n</code> 位小数
<code>sorted(iterable)</code>	返回新排序列表——不改变原序列
<code>enumerate(seq)</code>	返回 (索引, 值) 对
<code>zip(a, b)</code>	并行遍历多个序列
<code>map(func, iterable)</code>	对每个元素应用函数
<code>filter(func, iterable)</code>	保留 <code>func</code> 返回 <code>True</code> 的元素
<code>any(iterable)</code>	任一为 <code>True</code> 则 <code>True</code>
<code>all(iterable)</code>	全部为 <code>True</code> 才 <code>True</code>
<code>isinstance(obj, type)</code>	类型检查（支持继承判断）

8 Quick Reference Card / 速查卡

Top 20 Most Used Patterns / 最常用 20 条

1. `print(f"{var}")` —f-string 输出
2. `name = input("Prompt: ")` —获取用户输入
3. `int(x)` / `float(x)` / `str(x)` —类型转换
4. `len(seq)` —获取长度
5. `s[i]` / `s[i:j]` / `s[::-1]` —索引 / 切片 / 反转
6. `s.strip()` / `.split(d)` / `d.join(lst)` —字符串处理三件套
7. `if c1: ... elif c2: ... else: ...` —条件分支
8. `for item in iterable: ...` —for 循环遍历
9. `for i in range(n): ...` —固定次数循环
10. `while cond: ...` —条件循环（注意更新计数器）
11. `break` / `continue` / `pass` —循环控制
12. `def f(a, b=0): return x` —函数定义
13. `lambda x: x * 2` —匿名函数
14. `lst.append(x)` / `.pop()` / `.sort()` —列表核心操作
15. `lst.copy()` / `lst[:]` —浅拷贝（避免引用陷阱）
16. `d.get(key, default)` —字典安全取值
17. `for k, v in d.items(): ...` —字典遍历
18. `set(lst)` —列表去重
19. `[expr for x in lst if cond]` —列表推导式
20. `sorted(lst, key=..., reverse=True)` —自定义排序

Practice Makes Perfect / 熟能生巧

This tutorial covers the core Python syntax needed for the OOP course (22EM31202C).

For more advanced topics (NumPy, Pandas, Matplotlib, OOP, File I/O, Error Handling),

refer to the full **OOP.pdf** course notes and the **Command.pdf** quick reference.

本教程涵盖 OOP 课程 (22EM31202C) 所需的 Python 核心语法。

更进阶的内容 (NumPy、Pandas、Matplotlib、面向对象、文件 IO、异常处理)

请参考完整版 **OOP.pdf** 课程笔记和 **Command.pdf** 速查表。
